UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

Análise Comparativa de Desempenho entre PostgreSQL com JSONB e MongoDB para Catálogos com Dados Parcialmente Variáveis no Contexto de Delivery de Alimentos

Rauany Martinez Secci



São Carlos – SP

Análise Comparativa de Desempenho entre PostgreSQL com JSONB e MongoDB para Catálogos com Dados Parcialmente Variáveis no Contexto de Delivery de Alimentos

Rauany Martinez Secci

Orientador: **Profa. Dra. Elaine Parros Machado de Sousa**

Monografia final de conclusão de curso do Departamento de Sistemas de Computação do Instituto de Ciências Matemáticas e de Computação – ICMC-USP, para obtenção do título de Bacharel em Engenharia de Computação.
Área de Concentração: Sistemas de Banco de Dados.

USP – São Carlos
Novembro de 2026

Secci, Rauany Martinez
Análise Comparativa de Desempenho entre PostgreSQL com JSONB e MongoDB para Catálogos com Dados Parcialmente Variáveis no Contexto de Delivery de Alimentos / Rauany Martinez Secci. - São Carlos - SP, 2026.
67 p.; 29,7 cm.

Orientador: Elaine Parros Machado de Sousa.
Monografia (Graduação) - Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos - SP, 2026.

1. PostgreSQL. 2. MongoDB. 3. JSONB. 4. Dados semiestruturados. 5. Indexação. I. Sousa, Elaine Parros Machado de. II. Instituto de Ciências Matemáticas e de Computação (ICMC/USP). III. Título.

Dedico esse trabalho à minha família, por fazer com que meu sonho de estudar na USP fosse real, me dando todo o suporte e amor. Aos meus amigos, que tornaram essa fase a mais especial da minha vida. E ao meu namorado, por ter sido meu companheiro do início ao fim da graduação.

AGRADECIMENTOS

"É a educação que faz o futuro parecer um lugar de esperança e transformação"

(Marianna Moreno)

RESUMO

SECCI, R. M.. **Análise Comparativa de Desempenho entre PostgreSQL com JSONB e MongoDB para Catálogos com Dados Parcialmente Variáveis no Contexto de Delivery de Alimentos**. 2026. 67 f. Monografia (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

Observação 1: Esta seção é obrigatória. Observação 2: O resumo deve apresentar de forma concisa os pontos relevantes do texto. Deve descrever, de maneira objetiva e sucinta, o propósito da monografia, o objetivo do trabalho, sua relevância para a área de pesquisa onde está inserido, a metodologia empregada, e os resultados obtidos, com isso estimular a consulta ao texto completo da monografia. Observação 3: Deve ser redigido em parágrafo único, através de uma seqüência de frases concisas e objetivas e com espaçamento duplo. Observação 4: O resumo deve ter em torno de 15 linhas. Observação 5: Mais informações sobre a escrita de resumo, consulte a norma NBR 6028/1990 para escrita de resumos.

Palavras-chave: PostgreSQL, MongoDB, JSONB, Dados semiestruturados, Indexação.

ABSTRACT

SECCI, R. M.. **Análise Comparativa de Desempenho entre PostgreSQL com JSONB e MongoDB para Catálogos com Dados Parcialmente Variáveis no Contexto de Delivery de Alimentos**. 2026. 67 f. Monografia (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

Abstract é opcional

Key-words: Keyword 1, Keyword 2, Keyword 3, Keyword 4, Keyword 5.

LISTA DE ILUSTRAÇÕES

Figura 1 – Metodologia Geral do Desenvolvimento. 31
Figura 2 – Modelo de dados no PostgreSQL. 34
Figura 3 – Modelo de documentos no MongoDB. 34
Figura 4 – Hierarquia conceitual dos itens por categoria. 35

LISTA DE TABELAS

Tabela 1 – Principais bibliotecas Python e versões utilizadas. 33
Tabela 2 – Atributos da tabela `restaurantes` 35
Tabela 3 – Atributos da tabela `itens` 36
Tabela 4 – Estrutura do atributo `preco`. 36
Tabela 5 – Estrutura do atributo `opcoes_grupos`. 37
Tabela 6 – Estrutura do atributo `atributos_variaveis`. 37
Tabela 7 – Comparação entre os modelos do PostgreSQL e do MongoDB 39
Tabela 8 – Geração dos atributos dos restaurantes 40
Tabela 9 – Geração dos atributos dos itens 40
Tabela 10 – Atributos variáveis gerados por categoria 41
Tabela 11 – Opções geradas em cada grupo 42
Tabela 12 – Resumo das consultas implementadas 45
Tabela 13 – Combinações de consulta e índice avaliadas. 51
Tabela 14 – Volumes de dados avaliados. 57
Tabela 15 – Parâmetros variados no experimento de seletividade. 57
Tabela 16 – Operações avaliadas no experimento de lote de escrita. 58

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Criação das tabelas no PostgreSQL. 37
Código-fonte 2 – Cálculo da impressão digital do item. 42
Código-fonte 3 – Inserção dos dados no PostgreSQL. 43
Código-fonte 4 – Inserção dos dados no MongoDB. 44
Código-fonte 5 – C1 no PostgreSQL. 45
Código-fonte 6 – C1 no MongoDB. 46
Código-fonte 7 – C2 no PostgreSQL. 46
Código-fonte 8 – C2 no MongoDB. 47
Código-fonte 9 – C3 no PostgreSQL. 47
Código-fonte 10 – C3 no MongoDB. 47
Código-fonte 11 – C4 no PostgreSQL. 47
Código-fonte 12 – C4 no MongoDB. 48
Código-fonte 13 – C5 no PostgreSQL. 48
Código-fonte 14 – C5 no MongoDB. 48
Código-fonte 15 – C6 no PostgreSQL. 49
Código-fonte 16 – C6 no MongoDB. 49
Código-fonte 17 – C7 no PostgreSQL. 49
Código-fonte 18 – C7 no MongoDB. 49
Código-fonte 19 – C8 no PostgreSQL. 50
Código-fonte 20 – C8 no MongoDB. 50
Código-fonte 21 – Função de medição de tempo. 52
Código-fonte 22 – Funções de medição de recursos durante a inserção. 53
Código-fonte 23 – Funções de medição de espaço em disco. 55

SUMÁRIO

1 INTRODUÇÃO 21
1.1 Contextualização e Motivação 21
1.2 Objetivos 22
1.3 Organização do Trabalho 23

2 REVISÃO BIBLIOGRÁFICA 25
2.1 Conceitos e Técnicas Relevantes 25
2.1.1 Bancos Relacionais e Bancos Não Relacionais *25*
2.1.2 JSON vs JSONB *26*
2.1.3 PostgreSQL com JSONB *26*
2.1.4 MongoDB *27*
2.2 Trabalhos Relacionados 28
2.3 Considerações Finais 29

3 DESENVOLVIMENTO 31
3.1 Descrição do Problema e/ou Projeto 31
3.2 Descrição das Atividades Realizadas 32
3.2.1 Ambiente e Ferramentas Utilizadas *32*
3.2.2 Modelagem dos Dados e Schema *33*
3.2.2.1 Definição dos Atributos (PostgreSQL) *35*
3.2.2.2 Atributos JSONB *36*
3.2.2.3 Criação das tabelas no PostgreSQL *37*
3.2.2.4 Modelagem no MongoDB *38*
3.2.2.5 Limpeza do banco *39*
3.2.3 Geração dos Dados *39*
3.2.3.1 Restaurantes *39*
3.2.3.2 Itens *40*
3.2.4 Inserção dos dados *43*
3.2.5 Consultas Implementadas *44*
3.2.5.1 C1: itens por categoria e status. *45*
3.2.5.2 C2: upsert condicional. *46*
3.2.5.3 C3: exclusão lógica. *47*
3.2.5.4 C4: itens de um restaurante. *47*

3.2.5.5 C5: itens por faixa de preço. . . . 48
3.2.5.6 C6: valor de atributo variável aninhado. . . . 49
3.2.5.7 C7: filtro em array de objetos aninhados. . . . 49
3.2.5.8 C8: atualização parcial de JSONB. . . . 50
3.2.6 Estratégias de Indexação . . . ***50***
3.2.7 Metodologia de Medição . . . ***52***
3.2.7.1 Medição de tempo . . . 52
3.2.7.2 Medição de recursos . . . 53
3.2.7.3 Medição de espaço em disco . . . 55
3.2.8 Experimentos de Escalabilidade . . . ***56***
3.2.8.1 Volume total do banco . . . 57
3.2.8.2 Seletividade . . . 57
3.2.8.3 Lote de escrita . . . 58
3.3 Resultados Obtidos . . . **59**
3.3.1 Efeito do Volume e do Índice . . . ***59***
3.3.1.1 Consultas por id . . . 59
3.3.1.2 Consultas Seletivas . . . 59
3.3.1.3 Consultas pouco Seletivas . . . 59
3.3.1.4 Síntese do Ganho do Índice . . . 59
3.3.2 Seletividade . . . ***59***
3.3.3 Lote . . . ***59***
3.3.4 Carga Inicial dos Dados . . . ***59***
3.3.5 Armazenamento . . . ***59***
3.4 Dificuldades e Limitações . . . **59**
3.5 Considerações Finais . . . **60**

4 CONCLUSÃO . . . **61**

REFERÊNCIAS . . . **63**

APÊNDICE A EXEMPLO DE APÊNDICE . . . **65**

ANEXO A TÍTULO DO ANEXO . . . **67**

Capítulo 1

INTRODUÇÃO

Este capítulo apresenta o contexto em que o trabalho se insere, a motivação para a comparação entre PostgreSQL com JSONB e MongoDB, os objetivos da pesquisa e a forma como o texto está organizado.

1.1 Contextualização e Motivação

O crescimento do volume de dados semiestruturados trouxe novos desafios aos Sistemas de Gerenciamento de Bancos de Dados (SGBDs) relacionais, o que leva a adoção de SGBDs NoSQL (ELDAHSHAN *et al.*, 2022). Entre eles, os orientados a documentos, como o MongoDB[1], ganharam popularidade com o uso do formato JSON em aplicações web, pois armazenam dados semiestruturados de forma nativa, com esquema flexível e permitindo que documentos de uma mesma coleção tenham atributos distintos. No entanto, carecem de recursos dos sistemas relacionais (CHASSEUR; LI; PATEL, 2013), como junções eficientes, chaves estrangeiras para manter a consistência entre registros relacionados e transações ACID (atomicidade, consistência, isolamento e durabilidade) envolvendo múltiplas entidades. Por sua vez, o PostgreSQL[2] passou a incorporar suporte a dados semiestruturados em formatos como JSON, com esforços para evoluir esse suporte para representações binárias e indexáveis mais adequadas a esse modelo de dados (BARTUNOV; SIGAEV, 2013).

Essa discussão é relevante em catálogos de produtos, cujos dados costumam ser heterogêneos. Por exemplo, pode-se ter em um mesmo catálogo uma pizza que possui opções de borda e tamanhos, uma bebida que possui volume e informação de temperatura e uma sobremesa que possui informação nutricional específica, ou seja, cada tipo de produto tem suas próprias particularidades. A representação horizontal tradicional, com uma coluna por atributo, é inadequada (AGRAWAL; SOMANI; XU, 2001), pois a maioria das colunas ficaria vazia para a maioria dos itens, e cada nova categoria de produto exigiria alterar o esquema da tabela. Criar uma tabela para cada tipo de produto também não é viável, pois listar o cardápio completo passa a exigir a junção de várias tabelas, algo que seria muito custoso. Uma alternativa é manter os atributos comuns em colunas relacionais e armazenar a parte variável em uma coluna JSONB[3].

[1] <https://www.mongodb.com/docs/manual/>
[2] <https://www.postgresql.org/docs/current/>
[3] <https://www.postgresql.org/docs/current/datatype-json.html>

Outra alternativa é abandonar o modelo relacional e armazenar todo o catálogo em um SGBD orientado a documentos, como o MongoDB.

Esse é o cenário da equipe de Catálogo do iFood[4], empresa em que a autora realiza estágio. O serviço responsável por esse catálogo armazena os itens em uma tabela do PostgreSQL com 55 colunas, das quais 11 são do tipo JSONB, que armazenam objetos como preços, grupos de opções, horários, classificações, entre outros. Os atributos comuns a todos os itens permanecem em colunas relacionais, enquanto os que variam ficam em colunas JSONB, o que permite lidar com a variabilidade sem abrir mão da integridade transacional do PostgreSQL.

O JSONB resolve o problema de modelagem, mas o principal gargalo passa a ser o desempenho. Em sistemas relacionais, a ausência de um esquema fixo no JSONB resulta em consultas analíticas lentas (DURNER; LEIS; NEUMANN, 2021). No PostgreSQL, acessar um campo dentro de um JSONB exige operadores e índices específicos, como o *Generalized Inverted Index* (GIN)[5], enquanto no MongoDB o acesso a um campo do documento é nativo. A pergunta que motiva este trabalho é se o MongoDB seria uma alternativa mais adequada para o padrão de dados de um catálogo como o do iFood, ou o PostgreSQL com JSONB e índices oferece desempenho comparável.

Este trabalho é um recorte desse problema. Por se tratar de um sistema de produção que atende milhões de transações diárias, o iFood não permite a replicação de sua infraestrutura e de seus dados reais para fins de pesquisa acadêmica. Por isso, utiliza-se dados gerados sinteticamente, em escala controlada de até 1 milhão de itens, e projeta o problema em um cenário de dados estruturados e semiestruturados. Das 11 colunas JSONB do sistema real, 3 foram selecionadas, cada uma representando um padrão estrutural observado nas 11 colunas: um conjunto de campos fixos (igual para todos os itens do catálogo), um conjunto de campos que varia conforme a categoria do item, ou seja, o tipo de item vendido, como pizza, bebida ou sobremesa. Por fim, um *array* de estruturas aninhadas, em que cada elemento possui seus próprios campos.

1.2 Objetivos

O principal objetivo deste trabalho é comparar o desempenho do PostgreSQL, com a ferramenta JSONB, e do MongoDB, identificando em quais cenários cada tecnologia apresenta melhores resultados.

Para isso, são avaliadas estratégias de indexação genéricas e específicas em cada sistema, aplicadas a operações de leitura e escrita sobre dados estruturados e semiestruturados. Além disso, analisa-se a escalabilidade de cada tecnologia conforme a seletividade das consultas, o volume de dados e o tamanho do lote de escrita. Para garantir uma comparação justa, os

[4] <https://www.ifood.com.br/>
[5] <https://www.postgresql.org/docs/current/gin.html>

experimentos são executados em um ambiente controlado e com medição de tempo uniforme nos dois sistemas.

1.3 Organização do Trabalho

Este trabalho de conclusão de curso estrutura-se em quatro capítulos, dos quais este é o Capítulo 1. No Capítulo 2, é apresentada a fundamentação teórica dos principais conceitos e tecnologias empregadas neste trabalho, como PostgreSQL, JSONB, MongoDB, Índices, entre outros, juntamente com os trabalhos relacionados a este estudo. Em seguida, no Capítulo 3, descreve-se a metodologia adotada, detalhando-se a implementação das duas aplicações de catálogo comparadas e os resultados obtidos em todos os experimentos. Por fim, no Capítulo 4, são apresentadas as considerações finais sobre os resultados alcançados, as contribuições do trabalho e propostas para trabalhos futuros.

Capítulo 2

REVISÃO BIBLIOGRÁFICA

Neste capítulo são apresentados os conceitos fundamentais e o levantamento bibliográfico para a compreensão do problema da pesquisa. São discutidos os conceitos sobre os paradigmas relacional e não relacional (NoSQL), comparação técnica entre os formatos `JSON` e `JSONB` no PostgreSQL e as especificidades dos bancos de dados PostgreSQL e MongoDB. Por fim, apresenta uma análise dos principais trabalhos que servem como base para a solução desenvolvida, contextualizando como estudos anteriores abordaram comparações entre esses sistemas e identificando as lacunas que o presente trabalho se propõe a preencher.

2.1 Conceitos e Técnicas Relevantes

2.1.1 Bancos Relacionais e Bancos Não Relacionais

A evolução dos sistemas de armazenamento de dados é marcada pela busca por eficiência, integridade e escalabilidade diante do volume crescente e da variedade de informações. Assim, duas principais abordagens atendem requisitos de aplicações modernas: bancos de dados relacionais e não relacionais, logo, compreender as características de cada paradigma é essencial para a escolha arquitetural em sistemas como catálogos dinâmicos.

Sistemas Gerenciadores de Bancos de Dados Relacionais (SGBDR) organizam os dados em estruturas tabulares com esquema predefinido, o que favorece o tratamento de dados estruturados (MATCHA; MISHRA, 2025). Nesses sistemas, a confiabilidade transacional é sustentada pelas propriedades ACID: atomicidade, consistência, isolamento e durabilidade, que asseguram a execução íntegra das transações, a preservação de invariantes do sistema, a não interferência entre transações concorrentes e a persistência dos efeitos após falhas (WATERPEACE; IVANOV, 2026). Por essa razão, bancos relacionais permanecem amplamente utilizados em domínios críticos e permitem consultas complexas envolvendo múltiplas tabelas.

Bancos de dados não relacionais (NoSQL) surgiram como resposta às limitações dos sistemas relacionais tradicionais diante das crescentes demandas de armazenamento e consulta de grandes volumes de dados (DAVOUDIAN; CHEN; LIU, 2018). O paradigma NoSQL abrange diferentes categorias conforme a organização dos dados, como chave-valor, documentos, bancos orientados a colunas e grafos. Entre essas, os bancos orientados a documentos destacam-se por permitir que atributos dinâmicos e heterogêneos coexistam no mesmo documento sem estruturas

previamente fixadas (*schema-on-read*). Contudo, a literatura não trata o NoSQL como substituto universal do modelo relacional, mas como uma alternativa complementar (NAYAK; PORIYA; POOJARY, 2013).

2.1.2 JSON vs JSONB

O PostgreSQL oferece dois tipos de dados nativos para o armazenamento de documentos no formato JSON: JSON e JSONB. JSON armazena dados em formato textual, preservando a representação original do documento inserido. Em contraste, JSONB armazena dados em formato binário, processando o documento durante a inserção para otimizar seu armazenamento e acesso (LIU *et al.*, 2020).

A escolha por JSONB em relação a JSON é fortemente justificada pelas implicações de desempenho. Embora JSONB demande processamento adicional durante a inserção para realizar a conversão binária, esse custo é compensado significativamente nas operações de leitura e indexação.

Estudos indicam que o tipo de dado JSONB, por armazenar documentos em formato binário, oferece melhor desempenho em consultas, com tempos de execução menores em diferentes cenários avaliados (AJI; UTAMI, 2026), e permite indexação eficiente via índices GIN, enquanto o formato textual de JSON requer processamento custoso durante operações de consulta (LIU *et al.*, 2020). Para catálogos dinâmicos com padrões de acesso centrados em leitura, essa vantagem de desempenho é crucial.

2.1.3 PostgreSQL com JSONB

O PostgreSQL é um sistema gerenciador de banco de dados relacional de código aberto que utiliza a linguagem SQL (*Structured Query Language*) para definição, manipulação e consulta dos dados. Os dados são armazenados em tabelas organizadas em páginas de 8 KB e para permitir acesso concorrente, adota o controle de concorrência multiversão (MVCC)[1]: leitores consultam versões anteriores dos registros enquanto escritores atualizam a versão corrente, de modo que um não bloqueia o outro (BONDUGULA, 2024). Para acelerar as consultas, o PostgreSQL oferece diferentes tipos de índice, como B-tree[2], *Generalized Search Tree* (GiST)[3], GIN e *Block Range Index* (BRIN)[4].

Nesse contexto, o tipo JSONB faz com que o PostgreSQL seja híbrido, onde atributos estáveis permanecem em colunas relacionais e atributos variáveis, como os de um catálogo dinâmico, ficam em uma coluna JSONB. A abordagem proposta busca combinar o desenvolvimento

[1] <https://www.postgresql.org/docs/current/mvcc-intro.html>
[2] <https://www.postgresql.org/docs/current/btree.html>
[3] <https://www.postgresql.org/docs/current/gist.html>
[4] <https://www.postgresql.org/docs/current/brin.html>

rápido com JSON com a consistência do modelo relacional e a eficiência do SQL (IRSHAD; YAN; MA, 2019).

Neste trabalho utiliza-se três tipos de índice:

- **Índice simples:** um índice B-tree sobre uma única coluna. É a estrutura padrão do PostgreSQL, com busca de complexidade logarítmica, e apresenta bom desempenho em consultas de seleção e atualização.
- **Índice composto:** um índice B-tree sobre múltiplas colunas, indicado para filtros que combinam vários atributos. A ordem das colunas importa, pois o índice só é plenamente aproveitado por consultas que filtram suas colunas iniciais.
- **Índice GIN (*Generalized Inverted Index*):** um índice invertido que mapeia cada chave e valor presentes nos documentos aos registros que os contêm, sendo o indicado para colunas JSONB.

2.1.4 MongoDB

O MongoDB é um banco de dados não relacional orientado a documentos de código aberto, projetado para oferecer alta disponibilidade, escalabilidade horizontal e flexibilidade de esquema. Em vez de tabelas e linhas, o MongoDB organiza os dados em *coleções* compostas por *documentos*. Internamente, os documentos são armazenados no formato BSON (*Binary JSON*)[5], uma representação binária do JSON que organiza o documento como uma lista ordenada de elementos, cada um composto por nome do campo, tipo de dado e valor (TRUICĂ; BOICEA; TRIFAN, 2013).

Para o gerenciamento de armazenamento no disco, o MongoDB utiliza por padrão o motor *WiredTiger*[6] (NGUYEN; LEE, 2016). Esse motor emprega um mecanismo de concorrência no nível de documento com controle otimista (MongoDB, Inc., 2026), permitindo que múltiplas operações de leitura e escrita ocorram simultaneamente na mesma coleção sem bloquear a coleção inteira. Assim como no PostgreSQL, a eficiência no acesso aos documentos e a execução de consultas no MongoDB dependem diretamente do uso adequado de índices, destacando-se quatro tipos principais utilizados neste trabalho:

- **Índice de Campo Único (*Single Field Index*)[7]:** estrutura de busca baseada em árvore sobre um único campo de nível superior ou sobre um caminho específico dentro de um documento, com busca de complexidade logarítmica.

[5] <https://www.mongodb.com/docs/manual/reference/bson-types/>
[6] <https://www.mongodb.com/docs/manual/core/wiredtiger/>
[7] <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-single/>

- **Índice Composto (*Compound Index*)[8]:** estrutura de busca baseada em árvore sobre múltiplos campos de um mesmo documento. A sequência dos campos na definição do índice é fundamental, assim como no índice composto do PostgreSQL.
- **Índice Chave-Múltipla (*Multikey Index*)[9]:** estrutura de busca baseada em árvore, acionada automaticamente pelo MongoDB quando o campo indexado contém um *array*, permitindo que consultas filtrem com eficiência documentos que possuam valor dentro de coleções ou chaves aninhadas.
- **Índice Curinga (*Wildcard Index*)[10]:** indexa dinamicamente todos os campos e subdocumentos presentes na coleção (ou em um caminho específico). É a estrutura genérica indicada para cenários com atributos altamente heterogêneos e imprevisíveis.

2.2 Trabalhos Relacionados

A literatura recente investiga de forma ampla o desempenho comparativo entre sistemas de gerenciamento de bancos de dados relacionais e NoSQL. Para fundamentar a análise proposta neste trabalho, destacam-se os seguintes estudos relevantes.

Ortíz, Pierre e León (2016) conduziram um estudo comparativo avaliando o desempenho do PostgreSQL com a introdução do tipo `JSONB` frente ao MongoDB em cargas de até 4 milhões de documentos. Analisando métricas de tamanho do banco de dados, tempo de carga e tempo de resposta em consultas de seleção, os autores observaram que o PostgreSQL apresentou tempos de inserção e armazenamento em disco altamente competitivos. Nas consultas de leitura, o MongoDB manteve-se mais rápido em seleções com ordenação, enquanto o PostgreSQL com `JSONB` superou o MongoDB em operações de agrupamento (*GROUP BY*). O estudo é fundamental por demonstrar a evolução do PostgreSQL em capacidades não relacionais, porém limitou-se a uma comparação genérica sem aprofundar em padrões estruturais específicos de JSONB ou em estratégias de indexação avançadas.

Asiminidis, Kokkonis e Kontogiannis (2018) investigaram as características de desempenho do PostgreSQL ao comparar a estrutura relacional tradicional contra colunas contendo dados semiestruturados nos formatos `JSON` e `JSONB`, utilizando um conjunto de dados de IoT com mais de 1 milhão de registros. Os autores observaram que a inserção em campos `JSON`/`JSONB` é mais rápida do que no esquema relacional devido à ausência de checagens estritas de tipo na ingestão. Ao comparar os formatos semiestruturados, enquanto o tipo `JSON` armazena uma cópia exata do texto de entrada, o `JSONB` armazena os dados em bytes decompostos. Isso permitiu ao `JSONB` superar o `JSON` nas seleções e agregações, concluindo que o `JSONB` representa o melhor meio-termo entre a flexibilidade do `JSON` e o desempenho de consulta do modelo relacional.

[8] <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-compound/>
[9] <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-multikey/>
[10] <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-wildcard/>

Urnikienė, Steponavičienė e Atanasov (2026) apresentaram uma análise comparativa focada em cenários de *e-commerce*, avaliando o desempenho de leitura do PostgreSQL e do MongoDB sob volumes de até 300 mil produtos. Os resultados demonstraram que o PostgreSQL executou consultas analíticas complexas mais rápido do que o MongoDB, mantendo maior estabilidade e menor desvio padrão nas medições. Por outro lado, o MongoDB apresentou uma eficiência de armazenamento em disco superior devido ao encapsulamento em documentos incorporados. Os autores concluíram que esquemas relacionais normalizados mantêm vantagens expressivas em cenários de *e-commerce* onde cargas analíticas dominam o sistema, ressaltando que a escolha do banco deve ser guiada pela complexidade das consultas e volume de dados.

Estes trabalhos, embora fundamentais para evidenciar a diferença entre SGBDs relacionais e NoSQL, apresentam limitações para o cenário de catálogos dinâmicos de grande porte como o do iFood. Entre elas, destacam-se o tratamento genérico do `JSONB`, sem distinguir como a estrutura interna dos documentos afeta o desempenho, a exploração limitada da indexação, sem comparar índices simples, compostos e GIN no PostgreSQL com os índices equivalentes do MongoDB, e a ausência de uma análise de como o desempenho varia com a escalabilidade. O presente estudo diferencia-se ao propor uma análise de um recorte do cenário real, categorizando os atributos flexíveis do catálogo em três padrões estruturais de `JSONB`, utilizando diferentes tipos de indexação, além de quantificar o comportamento do PostgreSQL e do MongoDB isoladamente sob três eixos de escalabilidade em ambiente de contêineres Docker[11].

2.3 Considerações Finais

Este capítulo estabeleceu a fundamentação teórica necessária, apresentando os paradigmas relacional e não relacional, o tipo `JSONB` do PostgreSQL e o MongoDB, com seus respectivos tipos de índice. A revisão dos trabalhos relacionados evidenciou a lacuna existente na análise de padrões estruturais de `JSONB`, de estratégias de indexação e de escalabilidade em catálogos dinâmicos. O Capítulo 3 detalhará a metodologia adotada, abrangendo a modelagem e a geração dos dados, as consultas e os índices avaliados, bem como os experimentos de escalabilidade e os resultados obtidos.

[11] <https://docs.docker.com/>

Capítulo 3

DESENVOLVIMENTO

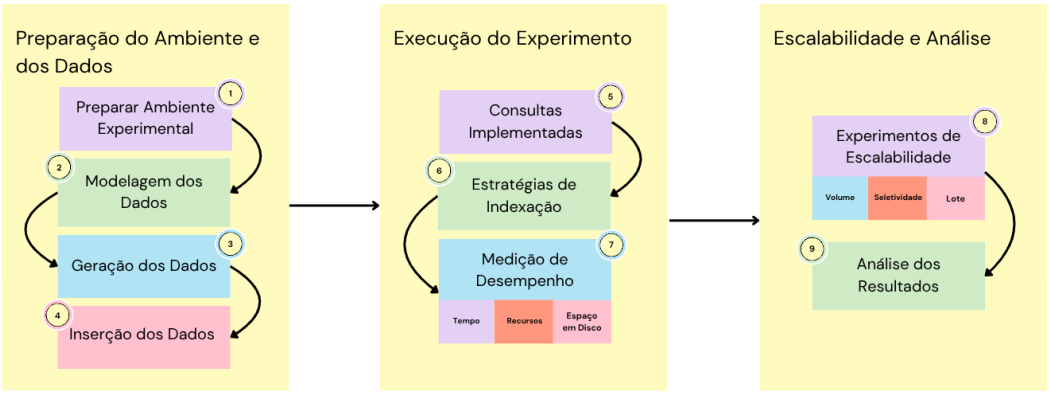
Este capítulo descreve o projeto desenvolvido: a comparação entre o PostgreSQL com JSONB e o MongoDB em um catálogo de dados parcialmente variáveis. A Seção 3.1 apresenta o problema e a metodologia, a Seção 3.2 detalha as atividades realizadas, a Seção 3.3 analisa os resultados e a Seção 3.4 discute as dificuldades e limitações.

3.1 Descrição do Problema e/ou Projeto

Sistemas de catálogo de restaurantes armazenam itens, opções e promoções cujas estruturas variam de um item para outro. Esses dados podem ser representados em um banco relacional com colunas JSONB, como o PostgreSQL, ou em um banco orientado a documentos, como o MongoDB. A pergunta que este trabalho busca responder é: em um sistema de catálogo com dados parcialmente variáveis, em quais cenários o PostgreSQL, com o recurso JSONB, e o MongoDB apresentam melhor desempenho e escalabilidade?

Para isso, foi implementado nos dois bancos um recorte controlado de um sistema de catálogo, com dados sintéticos e consultas que foram medidas em condições idênticas. A Figura 1 apresenta as etapas da metodologia, agrupadas em três fases.

Figura 1 – Metodologia Geral do Desenvolvimento.



Fonte: Elaborado pela autora.

Na **preparação do ambiente e dos dados** (etapas 1 a 4), os dois bancos são configurados em contêineres Docker, o modelo de dados é definido em cada um e os mesmos dados sintéticos

são gerados e inseridos. Na **execução do experimento** (etapas 5 a 7), um conjunto de consultas e operações de escrita é implementado no PostgreSQL e no MongoDB, avaliado com e sem índices e medido quanto ao tempo, ao consumo de recursos e ao espaço em disco. Por fim, na **escalabilidade e análise** (etapas 8 e 9), as operações são repetidas variando o volume de dados, a seletividade e o tamanho do lote de escrita, e os resultados são comparados. O detalhamento de cada etapa é apresentado na seção seguinte.

3.2 Descrição das Atividades Realizadas

3.2.1 Ambiente e Ferramentas Utilizadas

Todo o experimento foi executado localmente, em uma única máquina, com os dois bancos de dados em contêineres Docker. A implementação está disponível em duas formas: um notebook Jupyter, que apresenta cada etapa de forma explicativa, e um script Python (`analises.py`), que executa os experimentos de ponta a ponta e grava os resultados em arquivos JSON. O código completo, incluindo a implementação do experimento, o arquivo de configuração dos contêineres, o arquivo de geração de gráficos e lista de dependências, está disponível em um repositório público no GitHub[1].

A máquina utilizada possui processador Intel Core i5-9300H (2,40 GHz), 8 GB de memória RAM e sistema operacional Ubuntu 22.04.5 LTS. Os contêineres foram executados com Docker Engine, versão 29.1.3.

Os bancos foram definidos a partir de imagens com versão fixada: PostgreSQL 14.18 e MongoDB 8.0.29. Cada contêiner foi limitado a 2 CPUs e 3 GB de memória, o mesmo limite para os dois, garantindo condições equivalentes de execução. Os dados ficam em volumes persistentes, e as portas padrão de cada banco (5432 e 27017) são expostas ao notebook.

A implementação foi desenvolvida no Visual Studio Code[2] com Python[3] 3.10.12. O notebook e o script concentram as etapas do experimento: criação do esquema, geração e inserção dos dados, execução das consultas, criação dos índices, escalabilidade, medição de desempenho e gravação dos resultados. Antes de cada repetição de uma medição, os contêineres são reiniciados e o cache do sistema operacional é descartado, sem aquecimento, de modo que cada execução parte do cache vazio. As credenciais e os parâmetros de conexão não ficam no código: são lidos de arquivos `.env`.

A primeira etapa do script (e a primeira célula do notebook) realiza a configuração do ambiente, importa as bibliotecas, carrega as variáveis dos arquivos `.env`, cria a conexão com o

[1] <https://github.com/RauanySecci/TCC_Rauany>
[2] <https://code.visualstudio.com>
[3] <https://www.python.org>

PostgreSQL por meio do SQLAlchemy[4] e a conexão com o MongoDB por meio do PyMongo[5].

As versões de todas as dependências estão fixadas no arquivo `requirements.txt`, que permite recriar o ambiente Python e inclui também as dependências indiretas do Jupyter. As principais bibliotecas utilizadas estão na Tabela 1.

Tabela 1 – Principais bibliotecas Python e versões utilizadas.

Biblioteca	**Versão**	**Finalidade**
SQLAlchemy	2.0.53	Conexão e consultas ao PostgreSQL
psycopg2-binary	2.9.13	Driver do PostgreSQL e inserção em lotes (`execute_values`)
PyMongo	4.18.1	Conexão e consultas ao MongoDB
Faker	40.39.0	Geração de dados sintéticos
pandas	2.3.3	Manipulação dos resultados
matplotlib	3.10.9	Geração dos gráficos
numpy	2.2.6	Cálculos numéricos nos gráficos
python-dotenv	1.2.3	Leitura das variáveis do `.env`
`time` / `statistics`	3.10.12*	Medição (`perf_counter`) e agregação dos tempos
`subprocess` / `threading`	3.10.12*	Coleta de CPU e memória dos contêineres (cgroup, com `docker stats` como alternativa) durante a inserção dos dados

* Biblioteca padrão do Python, com a versão do interpretador. Fonte: Elaborado pela autora.

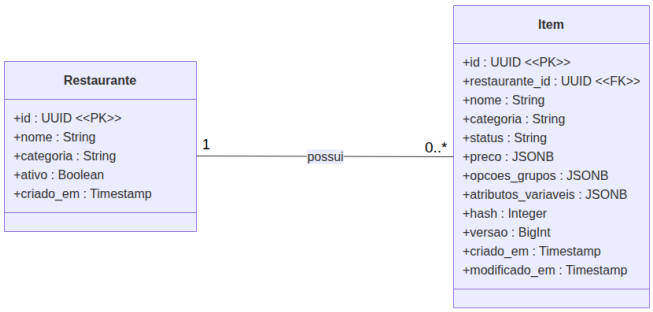
3.2.2 Modelagem dos Dados e Schema

Para representar o catálogo, o experimento utiliza duas entidades: restaurantes e itens. Essa escolha é um recorte do problema real, já que em um catálogo de produção há outras entidades e atributos, mas o objetivo aqui é avaliar o comportamento dos bancos sobre dados parcialmente variáveis, e não reproduzir o domínio completo. A tabela de itens é a principal do experimento, é nela que se concentram os atributos variáveis, o maior volume de registros e a maioria das consultas. A tabela de restaurantes existe para viabilizar a consulta que envolve junção, relacionando os itens ao restaurante que pertencem.

A Figura 2 apresenta o diagrama UML do modelo adotado no PostgreSQL, onde os dados fixos do item ocupam colunas tradicionais, e os dados que variam ocupam colunas JSONB.

[4] <https://www.sqlalchemy.org>
[5] <https://pymongo.readthedocs.io>

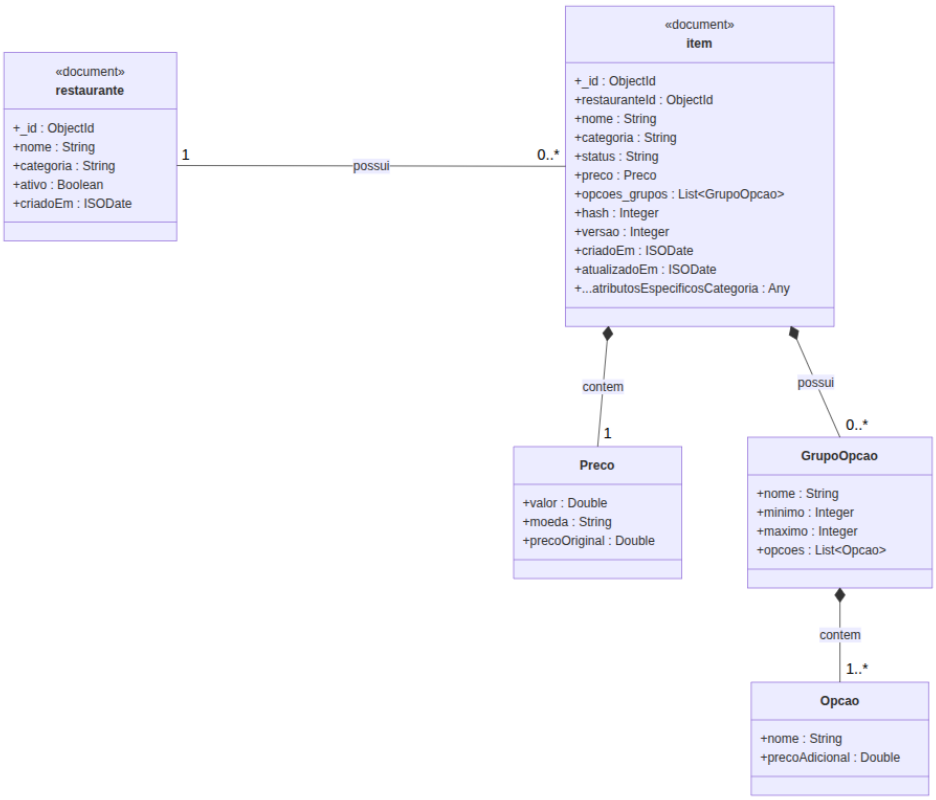
Figura 2 – Modelo de dados no PostgreSQL.



Fonte: Elaborado pela autora.

A Figura 3 apresenta o diagrama UML do modelo equivalente no MongoDB, onde restaurante e item são documentos. Item contém os objetos de preço e de grupos de opções e recebe os atributos variáveis como campos próprios (exemplificado por `atributosEspecificosCategoria`).

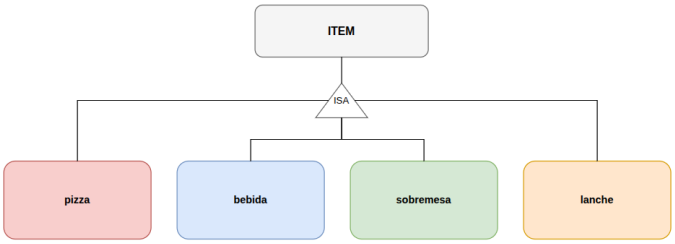
Figura 3 – Modelo de documentos no MongoDB.



Fonte: Elaborado pela autora.

A Figura 4 mostra a hierarquia conceitual dos itens, onde todo item pertence a uma das quatro categorias (pizza, bebida, sobremesa ou lanche). Essa hierarquia é conceitual e não é implementada como tabelas separadas, apenas reflete na estrutura do conteúdo JSONB.

Figura 4 – Hierarquia conceitual dos itens por categoria.



Fonte: Elaborado pela autora.

3.2.2.1 *Definição dos Atributos (PostgreSQL)*

A Tabela 2 apresenta os atributos da tabela `restaurantes`.

Tabela 2 – Atributos da tabela `restaurantes`

Atributo	**Tipo**	**Descrição**
`id`	`UUID`	Identificador único do restaurante (chave primária).
`nome`	`VARCHAR(255)`	Nome do restaurante.
`categoria`	`VARCHAR(100)`	Classificação da culinária do restaurante.
`ativo`	`BOOLEAN`	Indica se o restaurante está em funcionamento.
`criado_em`	`TIMESTAMP`	Data de criação do registro.

Fonte: Elaborado pela autora.

Já a Tabela 3 apresenta os atributos da tabela `itens`. Os atributos `hash` e `versao` não pertencem ao domínio do cardápio, eles são usados na lógica de inserção condicional, explicada na C2 da Seção 3.2.5.

Tabela 3 – Atributos da tabela `itens`

Atributo	Tipo	Descrição
`id`	`UUID`	Identificador único do item (chave primária).
`restaurante_id`	`UUID`	Restaurante a que o item pertence (chave estrangeira).
`nome`	`VARCHAR(255)`	Nome do item.
`categoria`	`VARCHAR(100)`	Pizza, bebida, sobremesa ou lanche.
`status`	`VARCHAR(20)`	`AVAILABLE`, `UNAVAILABLE` ou `DELETED`. O último representa a exclusão lógica, em que o registro permanece no banco.
`preco`	`JSONB`	Preço do item, em um objeto de estrutura fixa.
`opcoes_grupos`	`JSONB`	Grupos de opções configuráveis pelo cliente, em um array de objetos aninhados.
`atributos_variaveis`	`JSONB`	Características descritivas que variam conforme a categoria.
`hash`	`INT`	Valor derivado do conteúdo do item, usado para identificar se ele mudou.
`versao`	`BIGINT`	Contador de versão do item.
`criado_em`	`TIMESTAMP`	Data de criação do registro.
`modificado_em`	`TIMESTAMP`	Data da última modificação do registro.

Fonte: Elaborado pela autora.

3.2.2.2 *Atributos JSONB*

Os três atributos JSONB foram escolhidos para representar padrões estruturais distintos de dados semiestruturados, que se comportam de maneira diferente nas consultas e nos índices.

O atributo `preco` é um objeto simples, com estrutura igual para todos os itens (Tabela 4).

Tabela 4 – Estrutura do atributo `preco`.

Atributo	Tipo	Descrição	Obrigatório?
`valor`	Number (float)	Preço de venda atual do item.	Sim
`moeda`	String	Código da moeda corrente.	Sim
`precoOriginal`	Number (float) ou null	Preço antes do desconto. É preenchido apenas quando o item possui promoção, caso contrário, é `null`.	Não

Fonte: Elaborado pela autora.

O atributo `opcoes_grupos` é um array de objetos aninhados (Tabela 5). Cada grupo é um objeto com os campos `nome`, `minimo`, `maximo` (limites mínimo e máximo de escolhas) e `opcoes`.

Tabela 5 – Estrutura do atributo `opcoes_grupos`.

Categoria	Grupo esperado	Limites	Estrutura de `opcoes`	Obrigatório?
pizza	`Borda`	min: 0, max: 1	Array de {nome, precoAdicional}	Não
	`Tamanho`	min: 1, max: 1	Array de {nome, precoAdicional}	Sim
bebida	`Tamanho`	min: 1, max: 1	Array de {nome, precoAdicional}	Sim
sobremesa	`Acompanhamento`	min: 0, max: 2	Array de {nome, precoAdicional}	Não
lanche	`Adicionais`	min: 0, max: 5	Array de {nome, precoAdicional}	Não

Fonte: Elaborado pela autora.

O atributo `atributos_variaveis` é um objeto cujas chaves dependem da categoria do item (Tabela 6). Atributos como `tags` e `promocao` são opcionais e ocorrem em mais de uma categoria. Neste caso, atributos não obrigatórios são omitidos quando não se aplicam, em vez de armazenados como nulos. O campo `promocao` é um objeto com `descontoPercentual` (inteiro) e `validoAte` (texto com a data), e `informacaoNutricional` é um objeto com `calorias` (inteiro) e `contemGluten` (booleano).

Tabela 6 – Estrutura do atributo `atributos_variaveis`.

Categoria	Atributo esperado	Tipo	Obrigatório?
pizza	`promocao`	Objeto {descontoPercentual, validoAte}	Não
	`tags`	Array de strings	Não
bebida	`gelada`	Boolean	Sim
	`tags`	Array de strings	Não
sobremesa	`informacaoNutricional`	Objeto {calorias, contemGluten}	Sim
	`promocao`	Objeto {descontoPercentual, validoAte}	Não
lanche	`ingredientes`	Array de strings	Sim
	`tags`	Array de strings	Não
	`promocao`	Objeto {descontoPercentual, validoAte}	Não

Fonte: Elaborado pela autora.

3.2.2.3 *Criação das tabelas no PostgreSQL*

No PostgreSQL, o esquema é definido de forma explícita antes da inserção dos dados. O Código 1 apresenta os comandos de criação das duas tabelas. A chave estrangeira `fk_item_restaurante` garante a integridade referencial entre item e restaurante, e a cláusula `ON DELETE CASCADE` remove os itens de um restaurante quando ele é removido.

Código-fonte 1: Criação das tabelas no PostgreSQL.

```
CREATE TABLE restaurantes (
  id          UUID DEFAULT gen_random_uuid(),
  nome        VARCHAR(255) NOT NULL,
  categoria   VARCHAR(100),
  ativo       BOOLEAN NOT NULL DEFAULT true,
  criado_em   TIMESTAMP NOT NULL DEFAULT now(),
  CONSTRAINT pk_restaurante PRIMARY KEY (id)
);

CREATE TABLE itens (
  id                    UUID DEFAULT gen_random_uuid(),
  restaurante_id        UUID NOT NULL,
  nome                  VARCHAR(255) NOT NULL,
  categoria             VARCHAR(100) NOT NULL,
  status                VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
  preco                 JSONB NOT NULL,
  opcoes_grupos         JSONB,
  atributos_variaveis   JSONB NOT NULL DEFAULT '{}'::jsonb,
  hash                  INT,
  versao                BIGINT NOT NULL DEFAULT 1,
  criado_em             TIMESTAMP NOT NULL DEFAULT now(),
  modificado_em         TIMESTAMP NOT NULL DEFAULT now(),
  CONSTRAINT pk_item PRIMARY KEY (id),
  CONSTRAINT fk_item_restaurante FOREIGN KEY (restaurante_id)
    REFERENCES restaurantes(id) ON DELETE CASCADE
);
```

3.2.2.4 *Modelagem no MongoDB*

No MongoDB não há um esquema equivalente ao do PostgreSQL. As coleções `restaurantes` e `itens` não são declaradas previamente: elas passam a existir quando os primeiros documentos são inseridos, e a estrutura de cada documento é definida pelo próprio conteúdo inserido. Essa ausência de integridade referencial é uma característica dos bancos não relacionais discutida na Seção 2.1.1, e transfere para a aplicação a responsabilidade de manter a consistência entre item e restaurante.

A principal diferença estrutural está nos atributos variáveis. No PostgreSQL, eles ficam dentro da coluna `atributos_variaveis`, já no MongoDB eles são gravados como campos de nível superior do documento do item, ao lado de `nome` e `categoria`. Isso altera a forma de consultá-los: no PostgreSQL, o acesso parte da coluna `atributos_variaveis`, e no MongoDB, ele é feito diretamente pelo nome do campo no documento. Já `preco` e `opcoes_grupos` são, em ambos os bancos, um objeto e um array de objetos, respectivamente, mas no MongoDB são campos nativos do documento, e não valores de uma coluna.

A Tabela 7 resume as diferenças entre os dois modelos.

Tabela 7 – Comparação entre os modelos do PostgreSQL e do MongoDB

Aspecto	**PostgreSQL**	**MongoDB**
Esquema	Definido antes da inserção, com tipos e restrições	Definido pelo conteúdo inserido, sem validação
Identificador	`id` (UUID)	`_id` (ObjectId)
Referência ao restaurante	`restaurante_id`, com chave estrangeira	`restauranteId`, sem integridade referencial
Preço e opções	Colunas JSONB	Objeto e array embutidos no documento
Atributos variáveis	Coluna `atributos_variaveis`	Campos de nível superior do documento

Fonte: Elaborado pela autora.

3.2.2.5 Limpeza do banco

Antes de cada carga de dados, os dois bancos são limpos para que execuções anteriores não interfiram nos resultados. No PostgreSQL, as tabelas são removidas com `DROP TABLE IF EXISTS ... CASCADE` e recriadas com os comandos do Código 1. No MongoDB, as duas coleções são removidas com `drop()` e voltam a existir na inserção seguinte.

3.2.3 Geração dos Dados

Como não foi possível utilizar dados reais do catálogo que inspira este trabalho, os dados do experimento são gerados de forma sintética, por meio de um *script* em Python. A geração usa duas bibliotecas: a biblioteca padrão `random`, que realiza os sorteios, e a biblioteca `Faker`, configurada para o português do Brasil, que fornece sobrenomes e nomes de cidades para compor os nomes dos restaurantes. Os demais textos, como os nomes dos itens, dos ingredientes e das opções, vêm de listas definidas pela autora. As probabilidades descritas nesta seção também foram definidas de forma arbitrária, com o objetivo de tornar o catálogo heterogêneo.

Os mesmos dados gerados são inseridos nos dois bancos, de modo que a comparação ocorre sobre conteúdos idênticos. Apenas o formato do identificador muda: UUID no PostgreSQL e `ObjectId` no MongoDB. As sementes dos geradores aleatórios (`random.seed(42)` e `Faker.seed(42)`) são fixas, de modo que o conteúdo gerado se repete a cada execução. Os identificadores, por outro lado, não dependem dessas sementes e mudam a cada execução.

3.2.3.1 Restaurantes

A Tabela 8 resume como cada atributo do restaurante é gerado.

Tabela 8 – Geração dos atributos dos restaurantes

Atributo	**Geração**	**Distribuição**
Categoria	Atribuída de forma cíclica	Italiana, brasileira, árabe e mexicana, em partes iguais
Nome	Prefixo (três por categoria, como *Cantina*, *Trattoria* e *Pizzaria*) mais sobrenome ou nome de cidade do `Faker`	Único em todo o conjunto
Porte	Sorteio entre três portes, que define o peso do restaurante na distribuição dos itens	Pequeno (60%), médio (30%) e grande (10%)

Fonte: Elaborado pela autora.

Os pesos de porte fazem com que os restaurantes grandes concentrem, em média, cerca de 49% dos itens, os médios cerca de 37% e os pequenos cerca de 15%. Essa desigualdade é proposital, pois permite medir como a seletividade afeta uma consulta.

3.2.3.2 Itens

A Tabela 9 resume como cada atributo de Item é gerado.

Tabela 9 – Geração dos atributos dos itens

Atributo	**Geração**	**Distribuição**
Categoria	Sorteio com probabilidades distintas	Pizza: 35% Bebida: 20% Sobremesa: 10% Lanche: 35%
Nome	Nome-base (seis a oito por categoria, como *Margherita*, *Brownie* e *X-Bacon*) mais um complemento opcional (seis complementos possíveis ou nenhum, como *Tradicional* e *da Casa*)	Não precisa ser único
Status	Sorteio entre três valores	`AVAILABLE` (85%) `UNAVAILABLE` (10%) e `DELETED` (5%)
Atributos variáveis	Definidos por categoria, com sorteios independentes	Tabela 10
Opções de grupos	Definidas por categoria, com preço adicional fixo por opção	Tabela 11
Preço	Valor uniforme na faixa da categoria, com duas casas decimais	Pizza: R$ 35 a R$ 70 Bebida: R$ 5 a R$ 15 Sobremesa: R$ 12 a R$ 25 Lanche: R$ 18 a R$ 35

Fonte: Elaborado pela autora.

A distribuição das categorias foi definida para que o catálogo tivesse categorias de tamanhos diferentes, como ocorre em catálogos reais. Os itens com status `DELETED` existem porque o catálogo que inspira este trabalho realiza a exclusão por meio de *soft delete*.

Os atributos de `atributos_variaveis` são definidos por categoria, e cada atributo opcional é decidido por um sorteio independente. A lógica é: o gerador compara um número aleatório entre 0 e 1 com um limiar, o atributo só é incluído quando o número do sorteio o supera. Atributos que não se aplicam são omitidos do objeto. A Tabela 10 resume os atributos gerados e suas probabilidades.

Tabela 10 – Atributos variáveis gerados por categoria

Atributo	**Categorias**	**Presença**	**Conteúdo gerado**
`tags`	Pizza Bebida Lanche	Opcional	Sorteadas entre sete tags possíveis, cada uma com 30% de chance, então um item pode ter várias tags. Se nenhuma for sorteada, o atributo é omitido.
`promocao`	Pizza Sobremesa Lanche	Opcional: 40% 30% 35%	Objeto com `descontoPercentual` (inteiro entre 5 e 30 em 98% das promoções, e 50, 60, 70 ou 80 nos 2% restantes) e `validoAte` (data fixa).
`gelada`	Bebida	Sempre	Booleano, verdadeiro em 80% dos casos.
`informacao Nutricional`	Sobremesa	Sempre	Objeto com `calorias` (inteiro entre 200 e 500) e `contemGluten` (verdadeiro em 50% dos casos).
`ingredientes`	Lanche	Sempre	Lista com dois ou mais ingredientes, sorteados entre dez possíveis, cada um com 50% de chance, então um lanche pode ter vários ingredientes. Se saírem menos de dois, usa *queijo* e *bacon*.

Fonte: Elaborado pela autora.

Os grupos de `opcoes_grupos` são definidos por categoria, com os limites apresentados na Tabela 5. A Tabela 11 apresenta as opções de cada grupo e o preço adicional fixo de cada uma.

Tabela 11 – Opções geradas em cada grupo

Categoria	Grupo	Opção	Preço adicional	Presença
pizza	`Borda`	Sem borda	R$ 0,00	Sempre
		Borda mussarela	R$ 8,00	Sempre
		Borda catupiry	R$ 10,00	Sempre
	`Tamanho`	Média	R$ 0,00	Sempre
		Grande	R$ 12,00	Sempre
		Família	R$ 20,00	Sempre
bebida	`Tamanho`	350ml	R$ 0,00	Sempre
		500ml	R$ 2,00	Sempre
		600ml	R$ 3,50	Sempre
sobremesa	`Acompanhamento`	Calda	R$ 2,00	Sempre
		Sem acompanhamento	R$ 0,00	Sempre
lanche	`Adicionais`	Sem cebola	R$ 0,00	Sempre
		Bacon extra	R$ 3,00	Restaurantes que o oferecem (cerca de 60%)
		Cheddar extra	R$ 3,50	Restaurantes que o oferecem (cerca de 25%)
		Picles extra	R$ 2,50	Restaurantes pequenos que o oferecem (cerca de 1% dos restaurantes)

Fonte: Elaborado pela autora.

A oferta de *Bacon extra*, *Cheddar extra* e *Picles extra* é decidida no nível do restaurante, e não do item, porque não faria sentido um mesmo restaurante oferecê-los em alguns lanches e em outros não. Dessa forma, a diferença de oferta entre os restaurantes produz seletividades distintas nas consultas que filtram por essas opções.

O preço é um objeto, cujo campo `precoOriginal` só é preenchido quando o item possui promoção e é calculado a partir do percentual de desconto. Por esse motivo, os atributos variáveis são gerados antes do preço.

Cada item recebe um valor de `hash`, que funciona como uma impressão digital do seu conteúdo. Ele é calculado a partir do nome, da categoria, do status, do preço, das opções e dos atributos variáveis, serializados em JSON com as chaves ordenadas, o que garante que o mesmo conteúdo produza sempre o mesmo valor, o Código 2 apresenta esse cálculo. O objetivo é permitir, na inserção condicional, decidir se um item recebido difere do armazenado comparando apenas dois inteiros, em vez de comparar todos os campos.

Código-fonte 2: Cálculo da impressão digital do item.

```
1 def calcular_hash(nome, categoria, status, preco, opcoes_grupos,
      atributos):
```

```
2     conteudo = json.dumps(
3         {"nome": nome, "categoria": categoria, "status": status,
4          "preco": preco, "opcoes_grupos": opcoes_grupos, "atributos":
      atributos},
5         sort_keys=True, ensure_ascii=False
6     )
7     digest = hashlib.md5(conteudo.encode("utf-8")).hexdigest()
8     valor = int(digest[:8], 16)
9     if valor >= 2**31:
10        valor -= 2**32
11    return valor
```

3.2.4 Inserção dos dados

Os restaurantes são inseridos antes dos itens, porque os itens referenciam o restaurante. Os dois bancos recebem os itens em lotes de 10000, e cada lote é confirmado da seguinte forma: no PostgreSQL, com um `commit` ao final de cada lote, e no MongoDB com a opção `journal=True`. O banco gera o identificador de cada item (`DEFAULT gen_random_uuid()`) e preenche as datas de criação e modificação (`DEFAULT now()`). O Código 3 apresenta a inserção no PostgreSQL.

Código-fonte 3: Inserção dos dados no PostgreSQL.

```
1 execute_values(
2     cur,
3     "INSERT INTO restaurantes (id, nome, categoria, ativo) VALUES %s"
      ,
4     dados_restaurantes, page_size=10_000)
5 conn.commit()
6
7 for i in range(0, len(itens), 10_000):
8     rows = [(it["restaurante"]["pg_id"], it["nome"], it["categoria"],
9              it["status"],
10             json.dumps(it["preco"], ensure_ascii=False),
11             json.dumps(it["opcoes_grupos"], ensure_ascii=False),
12             json.dumps(it["atributos"], ensure_ascii=False),
13             it["hash"], it["versao"])
14            for it in itens[i:i + 10_000]]
15    execute_values(
16        cur,
17        "INSERT INTO itens (restaurante_id, nome, categoria, status,
      preco, "
18        "opcoes_grupos, atributos_variaveis, hash, versao) VALUES %s"
      ,
19        rows,
```

```
20          template="(%s, %s, %s, %s, %s::jsonb, %s::jsonb, %s::jsonb, %s, %s)",
21          page_size=10_000)
22      conn.commit()  # confirma a cada lote
```

No MongoDB, os itens são carregados com uma chamada a `insert_many` por lote, e o identificador do item é gerado pelo *driver*. Como o banco não preenche as datas, a aplicação as define com o instante do início da carga, o mesmo valor para todos os documentos. Além disso, os atributos variáveis são gravados diretamente no nível superior do documento (Código 4).

Código-fonte 4: Inserção dos dados no MongoDB.

```
1 agora = datetime.now()
2
3 docs_restaurantes = [{"_id": r["mongo_id"], "nome": r["nome"],
4                       "categoria": r["categoria"], "ativo": r["ativo"],
5                       "criado_em": agora} for r in restaurantes]
6 db_mongo.restaurantes.insert_many(docs_restaurantes)
7
8 for i in range(0, len(itens), 10_000):
9     docs = []
10    for it in itens[i:i + 10_000]:
11        doc = {"restauranteId": it["restaurante"]["mongo_id"],
12               "nome": it["nome"], "categoria": it["categoria"],
13               "status": it["status"], "preco": it["preco"],
14               "opcoes_grupos": it["opcoes_grupos"],
15               "hash": it["hash"], "versao": it["versao"],
16               "criado_em": agora, "modificado_em": agora}
17        doc.update(it["atributos"])  # atributos variaveis no nivel superior
18        docs.append(doc)
19    db_mongo.itens.insert_many(docs)
```

Durante cada inserção são medidos o tempo total, o uso de CPU e de memória do contêiner e o espaço ocupado pelos dados. Ao final da carga, o PostgreSQL executa o comando `ANALYZE` nas duas tabelas, para que todos os cenários partam das mesmas estatísticas, em vez de depender do momento em que o *autovacuum* rodaria sozinho.

3.2.5 Consultas Implementadas

As consultas do experimento foram definidas a partir de dois critérios. O primeiro é a recorrência: foram escolhidas operações que ocorrem com frequência no cenário do iFood, como carregar o cardápio de um restaurante, filtrar itens por categoria, processar eventos de atualização

do catálogo e remover itens do cardápio. O segundo é a investigação de possíveis gargalos: como a busca em arrays de objetos aninhados dentro de um campo JSONB e a busca por um valor específico dentro de um atributo opcional, como o valor da promoção, presente em apenas parte dos itens. Cada consulta representa um cenário distinto de uso do catálogo.

As oito consultas foram divididas em dois grupos, conforme o tipo de dado sobre o qual atuam. consultas sobre dados estruturados (C1 a C4) operam sobre campos escalares comuns. As consultas sobre dados semiestruturados (C5 a C8) operam sobre o conteúdo dos atributos JSONB. A Tabela 12 resume as consultas, e as seções seguintes apresentam cada uma com a implementação nos dois bancos.

Tabela 12 – Resumo das consultas implementadas

Id.	**Grupo**	**Operação**	**Padrão de acesso**	**Cenário representado**
C1	Estruturado	Leitura	Igualdade na categoria e exclusão de um status	Listar os itens não excluídos de uma categoria específica.
C2	Estruturado	Escrita	Upsert condicional por identificador	Processar um evento de atualização do catálogo.
C3	Estruturado	Escrita	*Soft delete* por identificador	Remover um item sem excluí-lo fisicamente.
C4	Estruturado	Junção	Junção e filtro de status	Carregar o cardápio completo de um restaurante.
C5	Semiestruturado	Leitura	Intervalo em campo numérico do JSONB	Filtrar itens por faixa de preço.
C6	Semiestruturado	Leitura	Igualdade em valor de atributo aninhado	Buscar itens em promoção.
C7	Semiestruturado	Leitura	Contenção em array aninhado	Buscar itens que oferecem uma opção.
C8	Semiestruturado	Escrita	Atualização parcial do JSONB por identificador	Adicionar uma tag a um item.

Fonte: Elaborado pela autora.

3.2.5.1 C1: itens por categoria e status.

Retorna os itens de uma categoria, por exemplo *pizza*, que não tenham sido excluídos logicamente. Essa consulta representa o cliente que busca um tipo de produto e recebe os itens dessa categoria. O padrão de acesso é a igualdade em `categoria` combinada com a exclusão de um valor de `status`, ambos campos escalares. O Código 5 apresenta a consulta no PostgreSQL, e o Código 6, no MongoDB.

Código-fonte 5: C1 no PostgreSQL.

```
1 SELECT nome, categoria, status
2 FROM itens
```

```
3 WHERE categoria = :categoria AND status <> 'DELETED';
```

Código-fonte 6: C1 no MongoDB.

```
1 db.itens.find(
2     {"categoria": categoria, "status": {"$ne": "DELETED"}},
3     {"_id": 0, "nome": 1, "categoria": 1, "status": 1}
4 )
```

3.2.5.2 C2: upsert condicional.

Representa a operação de escrita mais frequente de catálogo do iFood. Insere um item novo ou atualiza um já existente, aplicando a mudança apenas se o conteúdo realmente mudou (`hash` diferente) e se o evento não é mais antigo que o já processado (`versao` maior ou igual). No PostgreSQL, a decisão é tomada pelo próprio banco, em uma única instrução com uma expressão com um `UPDATE` condicionado a essas duas regras, seguido de um `INSERT` que só ocorre se o item ainda não existe (Código 7). Não foi usado o comando `INSERT ... ON CONFLICT`, que seria a forma mais direta, porque ele sempre usa o índice da chave primária para detectar o conflito e, assim, não permitiria medir a operação sem o índice, que será necessário para analisar mais adiante.

Código-fonte 7: C2 no PostgreSQL.

```
1 WITH atualizado AS (
2   UPDATE itens
3   SET nome = :nome, categoria = :categoria, status = :status,
4       preco = CAST(:preco AS JSONB),
5       opcoes_grupos = CAST(:opcoes_grupos AS JSONB),
6       atributos_variaveis = CAST(:atributos AS JSONB),
7       hash = :hash, versao = :versao, modificado_em = now()
8   WHERE id = CAST(:id AS UUID) AND hash <> :hash AND :versao >=
     versao
9   RETURNING id
10 )
11 INSERT INTO itens (id, restaurante_id, nome, categoria, status, preco
     , opcoes_grupos, atributos_variaveis, hash, versao)
12 SELECT CAST(:id AS UUID), CAST(:restaurante_id AS UUID), :nome, :
     categoria, :status, CAST(:preco AS JSONB), CAST(:opcoes_grupos AS
     JSONB), CAST(:atributos AS JSONB), :hash, :versao
13 WHERE NOT EXISTS (SELECT 1 FROM itens WHERE id = CAST(:id AS UUID));
```

O MongoDB não possui uma instrução única de upsert condicional equivalente à do PostgreSQL, de modo que a aplicação lê primeiro o documento existente e só então decide entre

inserir, substituir ou não fazer nada (Código 8).

Código-fonte 8: C2 no MongoDB.

```
existente = db.itens.find_one({"_id": evento["_id"]})
if existente is None:
    db.itens.insert_one(evento)
elif existente["hash"] != evento["hash"] and evento["versao"] >= existente["versao"]:
    evento["criado_em"] = existente["criado_em"]
    db.itens.replace_one({"_id": evento["_id"]}, evento)
# caso contrario, nao faz nada
```

3.2.5.3 C3: exclusão lógica.

Representa a remoção de um item do cardápio pelo restaurante, o sistema real não executa exclusões físicas, apenas lógicas. O padrão de acesso é a atualização do campo `status`, localizando o item pelo identificador. Além do `status`, a operação zera o `hash` e atualiza `modificado_em`. Os Códigos 9 e 10 apresentam a operação nos dois bancos.

Código-fonte 9: C3 no PostgreSQL.

```
UPDATE itens
SET status = 'DELETED', hash = 0, modificado_em = now()
WHERE id = ANY(CAST(:ids AS uuid[])) AND status <> 'DELETED';
```

Código-fonte 10: C3 no MongoDB.

```
db.itens.update_many(
    {"_id": {"$in": ids}, "status": {"$ne": "DELETED"}},
    {"$set": {"status": "DELETED", "hash": 0, "modificado_em": datetime.now()}}
)
```

3.2.5.4 C4: itens de um restaurante.

Retorna os itens de um restaurante específico, excluindo os removidos. O padrão de acesso é uma junção entre itens e restaurantes, com filtro de status, e a busca é feita pelo nome do restaurante. No PostgreSQL, a junção é resolvida pelo banco em uma única instrução (Código 11).

Código-fonte 11: C4 no PostgreSQL.

```
SELECT i.nome, i.categoria, i.status, r.nome AS restaurante_nome
```

```
2 FROM itens i
3 JOIN restaurantes r ON r.id = i.restaurante_id
4 WHERE r.nome = :nome AND i.status <> 'DELETED';
```

No MongoDB, a aplicação busca primeiro o restaurante pelo nome, para obter seu identificador, e em seguida consulta os itens que o referenciam (Código 12). Embora o MongoDB ofereça o operador `$lookup`, optou-se por duas consultas simples, que é a forma usual de resolver uma referência entre coleções.

Código-fonte 12: C4 no MongoDB.

```
1 restaurante = db.restaurantes.find_one({"nome": nome})
2 db.itens.find(
3     {"restauranteId": restaurante["_id"], "status": {"$ne": "DELETED"
  }},
4     {"_id": 0, "nome": 1, "categoria": 1, "status": 1}
5 )
```

3.2.5.5 *C5: itens por faixa de preço.*

Retorna os itens cujo preço está entre um valor mínimo e um máximo. Como o preço é armazenado dentro do atributo JSONB, o filtro precisa acessar o campo `valor` dentro do objeto. No PostgreSQL, o valor é extraído como texto com o operador `->>` e convertido para numérico (Código 13).

Código-fonte 13: C5 no PostgreSQL.

```
1 SELECT nome, categoria, preco
2 FROM itens
3 WHERE (preco->>'valor')::numeric BETWEEN :minimo AND :maximo
4   AND status <> 'DELETED';
```

No MongoDB, o campo interno é acessado diretamente por notação de ponto (Código 14).

Código-fonte 14: C5 no MongoDB.

```
1 db.itens.find(
2     {"preco.valor": {"$gte": minimo, "$lte": maximo},
3      "status": {"$ne": "DELETED"}},
4     {"_id": 0, "nome": 1, "categoria": 1, "preco": 1}
5 )
```

3.2.5.6 C6: valor de atributo variável aninhado.

Representa a busca por itens com uma característica rara dentro de um atributo opcional, como as promoções de desconto alto. O padrão de acesso é a igualdade em um valor aninhado: `promocao.descontoPercentual` igual a 80%. No PostgreSQL, usa-se o operador de contenção `@>` sobre o JSONB (Código 15), e no MongoDB, a notação de ponto sobre o campo aninhado (Código 16).

Código-fonte 15: C6 no PostgreSQL.

```
1 SELECT nome, categoria, atributos_variaveis->'promocao' AS promocao
2 FROM itens
3 WHERE atributos_variaveis @> CAST(:filtro AS JSONB)
4   AND status <> 'DELETED';
5 -- filtro: {"promocao": {"descontoPercentual": 80}}
```

Código-fonte 16: C6 no MongoDB.

```
1 db.itens.find(
2     {"promocao.descontoPercentual": 80, "status": {"$ne": "DELETED"
    }},
3     {"_id": 0, "nome": 1, "categoria": 1, "promocao": 1}
4 )
```

3.2.5.7 C7: filtro em array de objetos aninhados.

Representa a busca por itens que oferecem uma personalização específica, como *Bacon extra*, e envolve a estrutura mais complexa do modelo: um array de grupos, cada um com seu próprio array de opções, o que resulta em dois níveis de aninhamento. No PostgreSQL, usa-se o operador de contenção `@>` com um filtro JSONB que descreve a estrutura procurada (Código 17).

Código-fonte 17: C7 no PostgreSQL.

```
1 SELECT nome, categoria, opcoes_grupos
2 FROM itens
3 WHERE opcoes_grupos @> '[{"opcoes": [{"nome": "Bacon extra"}]}]'::
    jsonb AND status <> 'DELETED';
```

No MongoDB, a notação de ponto atravessa os dois arrays e compara o nome da opção diretamente (Código 18).

Código-fonte 18: C7 no MongoDB.

```
1 db.itens.find(
```

```
2     {"opcoes_grupos.opcoes.nome": "Bacon extra",
3      "status": {"$ne": "DELETED"}},
4     {"_id": 0, "nome": 1, "categoria": 1, "opcoes_grupos": 1}
5 )
```

3.2.5.8 C8: atualização parcial de JSONB.

Adiciona a tag *novidade* a um item específico. O padrão de acesso é a atualização de um campo interno do JSONB. No PostgreSQL, a tag é acrescentada à lista existente com `jsonb_set` e concatenação, usando `COALESCE` para o caso de o item ainda não ter tags. Como o operador `||` apenas concatena e duplicaria a tag em um item que já a possui, a cláusula `WHERE` seleciona somente os itens que ainda não contêm o valor (Código 19).

Código-fonte 19: C8 no PostgreSQL.

```
1 UPDATE itens
2 SET atributos_variaveis = jsonb_set(
3       atributos_variaveis, '{tags}',
4       COALESCE(atributos_variaveis->'tags', '[]'::jsonb) || '"
    novidade"'::jsonb
5     ),
6     modificado_em = now()
7 WHERE id = ANY(CAST(:ids AS uuid[]))
8   AND NOT COALESCE(atributos_variaveis->'tags', '[]'::jsonb) @> '"
    novidade"'::jsonb;
```

No MongoDB, o operador `$addToSet` acrescenta o valor ao array apenas se ele ainda não estiver presente. Para que as duas consultas sejam equivalentes, o filtro de ambas seleciona somente os itens que ainda não possuem a tag (Código 20). Assim, em ambos os bancos, a data de modificação só é atualizada quando a tag é de fato adicionada.

Código-fonte 20: C8 no MongoDB.

```
1 db.itens.update_many(
2     {"_id": {"$in": ids}, "tags": {"$ne": "novidade"}},
3     {"$addToSet": {"tags": "novidade"},
4      "$set": {"modificado_em": datetime.now()}}
5 )
```

3.2.6 Estratégias de Indexação

Esta seção descreve as estratégias de indexação avaliadas para cada consulta, cada uma foi executada primeiro sem índices adicionais e depois com os índices candidatos. Os índices

foram escolhidos conforme o padrão de acesso de cada consulta e os incompatíveis com esse padrão não foram testados. O procedimento é o mesmo para as duas tecnologias:

1. Antes de cada configuração, todos os índices que foram anteriormente criados pela autora são removidos. No PostgreSQL, é usada a função `apagar_indices_pg()`, que automatiza o comando `DROP INDEX`. No MongoDB, é usado `drop_indexes()`. Assim, nenhum índice de uma configuração anterior interfere na seguinte.
2. Logo em seguida o índice é criado. No PostgreSQL, a criação é seguida do comando `ANALYZE`, que atualiza as estatísticas da tabela, de modo que o plano escolhido reflita o índice testado e não a desatualização das estatísticas.
3. O plano de execução é consultado com `EXPLAIN` no PostgreSQL e com `explain` (modo `executionStats`) no MongoDB. Esse passo confirma se o índice criado foi escolhido pelo otimizador.
4. Quando o planejador do PostgreSQL ignora o índice criado e opta por uma varredura sequencial, a consulta é medida também com o uso do índice forçado (`enable_seqscan = off`), para separar o efeito do índice da decisão do planejador.
5. Fazemos duas medições sobre os índices, a primeira é em relação ao tempo de execução da consulta e a segunda ao espaço ocupado pelo índice. O procedimento de cada uma está descrito na Seção 3.2.7.

A Tabela 13 resume as combinações de consulta e índice testadas.

Tabela 13 – Combinações de consulta e índice avaliadas.

Consulta	**PostgreSQL**			**MongoDB**			
	Sem índice	**Simples/ Composto**	**GIN**	**Sem índice**	**Simples/ Composto**	**Multikey**	**Wildcard**
C1 — Categoria e status	✓	✓	–	✓	✓	–	–
C2 — Upsert condicional	✓[b]	✓[a]	–	✓[b]	✓[a]	–	–
C3 — Exclusão lógica	✓[b]	✓[a]	–	✓[b]	✓[a]	–	–
C4 — Itens de um restaurante	✓	✓	–	✓	✓	–	–
C5 — Faixa de preço	✓	✓	–	✓	✓	–	–
C6 — Valor de atributo aninhado	✓	–	✓	✓	–	–	✓
C7 — Array de objetos aninhados	✓	–	✓	✓	–	✓	–
C8 — Atualização parcial	✓[b]	✓[a]	–	✓[b]	✓[a]	–	–

[a] Utiliza apenas o índice da chave primária (criado automaticamente); [b] Varredura forçada da tabela; ✓: configuração testada; –: configuração não aplicável ao padrão de acesso. Fonte: Elaborado pela autora.

A escolha de cada tipo de índice, e o motivo de não usar os outros, foi a seguinte:

- **B-tree (simples ou composto)**: usado em C1, C4 e C5, que comparam valores escalares por igualdade ou intervalo, e em C2, C3 e C8, que utilizam apenas o índice da chave primária, criado automaticamente, para localizar o item pelo identificador. Em C6 e C7, as consultas usam o operador de contenção `@>` do JSONB, que o índice *B-tree* não suporta.
- **GIN**: usado em C6 e C7, que procuram dentro do JSONB. Não é usado de C1 a C4, pois os campos são escalares e não têm elementos internos a indexar. Também não é usado em C5, pois o GIN não atende a comparações de intervalo.
- ***Multikey***: usado em C7, sobre o caminho `opcoes_grupos.opcoes.nome`, que atravessa arrays. Não é usado no restante pois os campos consultados não são arrays.
- ***Wildcard***: usado em C6, em que o atributo consultado existe só em parte dos itens e um único índice indexa automaticamente todos os caminhos presentes em cada documento, dispensando a criação de um índice por atributo. Não é usado no restante, pois o campo consultado é sempre conhecido.

3.2.7 Metodologia de Medição

As medições foram implementadas em três funções independentes, cada uma dedicada a um certo tipo de desempenho: tempo de execução, consumo de recursos computacionais e espaço ocupado em disco. As mesmas funções foram aplicadas aos dois bancos, de modo que as diferenças observadas decorram do banco e não do método de medição.

3.2.7.1 Medição de tempo

Inicialmente, o tempo seria obtido a partir do próprio plano de execução, por meio do `EXPLAIN ANALYZE` no PostgreSQL e do `explain` no MongoDB. Essa abordagem se mostrou inadequada para as operações de escrita, já que no PostgreSQL o `EXPLAIN ANALYZE` executa o comando, enquanto no MongoDB o `explain` apenas simula a operação, sem aplicá-la. Logo, os tempos de escrita dos dois bancos não seriam comparáveis de maneira justa.

Optou-se então por cronometrar a operação do lado do cliente, com a função `time.perf_counter()` da biblioteca padrão do Python. A mesma função e os mesmos pontos de início e fim são usados para consultas, inserções, atualizações e exclusões nos dois bancos.

A função `medir_tempo`, apresentada no Código 21, recebe a ação a ser cronometrada e a executa repetidamente.

Código-fonte 21: Função de medição de tempo.

```
1 def medir_tempo(acao, repeats=10, warm_up=True, antes=None):
2     try:
3         if warm_up:
4             if antes:
```

```
5                 antes()
6             acao()
7         tempos = []
8         for _ in range(repeats):
9             if antes:
10                antes()
11            inicio = time.perf_counter()
12            acao()
13            tempos.append((time.perf_counter() - inicio) * 1000)
14        return {"min": min(tempos), "max": max(tempos),
15                "media": statistics.mean(tempos),
16                "mediana": statistics.median(tempos),
17                "tempos_brutos": tempos}
18    except Exception as e:
19        print("Erro ao executar benchmark:", e)
20        return None
```

Seus parâmetros e etapas são os seguintes:

- `acao`: função que executa a operação a ser medida (uma consulta, um *insert*, um *update*).
- `warm_up`: quando ativo, a ação é executada uma vez antes da medição e o resultado é descartado. Isso evita que o custo da primeira execução distorça as medidas. Nos experimentos deste trabalho esse parâmetro permanece desativado, pois o objetivo é medir justamente a execução com o cache vazio. Mas ele pode ser ativado em experimentos futuros para avaliar o impacto do aquecimento na performance.
- `antes`: função opcional executada antes de cada repetição e fora da cronometragem, usada quando é preciso restaurar o estado do banco entre as execuções.
- `repeats`: número de repetições cronometradas, igual a 10 em todas as medições.

Para cada ação, a função devolve o tempo mínimo, o máximo, a média e a mediana em milissegundos, além dos tempos brutos de cada repetição.

3.2.7.2 Medição de recursos

O consumo de CPU e de memória dos contêineres foi medido durante a inserção em massa dos dados. A leitura é feita diretamente nos contadores do cgroup do Linux (cgroup v2) do contêiner, que fornecem os valores instantaneamente. O comando `docker stats` foi mantido como alternativa, usada apenas quando o cgroup não é encontrado. O Código 22 apresenta as funções que trabalham em conjunto para capturar e monitorar esses recursos.

Código-fonte 22: Funções de medição de recursos durante a inserção.

```
def ler_cgroup(base):
    # CPU acumulada (s) e memoria (MiB), sem o cache de arquivos
    uso = next(int(l.split()[1]) for l in (base / "cpu.stat").
    read_text().splitlines()
                if l.startswith("usage_usec"))
    inativo = next(int(l.split()[1]) for l in (base / "memory.stat").
    read_text().splitlines()
                    if l.startswith("inactive_file"))
    memoria = int((base / "memory.current").read_text()) - inativo
    return uso / 1e6, memoria / 1048576

def monitorar_recursos(nome_container, amostras, parar, intervalo):
    base = caminho_cgroup(nome_container)
    cpu_ant, _ = ler_cgroup(base)
    t_ant = time.perf_counter()
    while not parar.is_set():
        parar.wait(intervalo)
        cpu_s, mem = ler_cgroup(base)
        agora = time.perf_counter()
        cpu = (cpu_s - cpu_ant) / (agora - t_ant) * 100
        cpu_ant, t_ant = cpu_s, agora
        if _enviando.is_set():
            amostras.append((cpu, mem))

def medir_insercao_com_recursos(funcao_insercao, nome_container,
    intervalo=0.25):
    amostras = []
    parar = threading.Event()
    monitor = threading.Thread(target=monitorar_recursos,
                               args=(nome_container, amostras, parar,
    intervalo))
    monitor.start()
    inicio = time.perf_counter()
    try:
        funcao_insercao()
    finally:
        tempo_total_ms = (time.perf_counter() - inicio) * 1000
        parar.set()
        monitor.join()
    cpus = [a[0] for a in amostras]
    memorias = [a[1] for a in amostras]
    return {"tempo_envio_ms": tempo_envio_s * 1000,
            "tempo_total_ms": tempo_total_ms,
            "cpu_media_percent": statistics.mean(cpus) if cpus else
    None,
            "cpu_max_percent": max(cpus) if cpus else None,
```

```
42              "memoria_media_mb": statistics.mean(memorias) if memorias
      else None,
43              "memoria_max_mb": max(memorias) if memorias else None,
44              "num_amostras": len(amostras),
45              "cpus_brutos": cpus, "memorias_brutas": memorias}
```

- `ler_cgroup`: lê o tempo de CPU acumulado pelo contêiner e a memória em uso. Da memória é descontado o cache de arquivos inativo, como faz o `docker stats`, de modo que o valor reflita a memória efetivamente utilizada.
- `monitorar_recursos`: a cada intervalo (0,25 s), calcula o percentual de CPU, expresso em porcentagem de um núcleo, e registra a memória, que já vem em MiB. Cada amostra, composta pelo par CPU e memória, só é armazenada se o envio de dados ao banco estiver em andamento, o que exclui o tempo gasto na conversão dos dados no Python.
- `medir_insercao_com_recursos`: dispara o monitor em uma *thread* separada, executa a inserção na *thread* principal, cronometra o tempo com o `perf_counter` e, ao final, sinaliza a parada do monitor. Devolve dois tempos, o de envio ao banco (`tempo_envio_ms`) e o total, que inclui a conversão dos dados lote a lote (`tempo_total_ms`), além da média e do máximo de CPU e de memória entre as amostras e das leituras brutas.

Na prática, a função tira uma "foto" do estado do contêiner a cada 0,25 s enquanto a inserção ocorre, e resume os recursos utilizados ao final.

3.2.7.3 Medição de espaço em disco

Além do tempo, o espaço ocupado é uma métrica relevante, pois um índice que acelera as consultas pode ter um custo significativo de armazenamento. As funções do Código 23 medem o tamanho dos dados e o dos índices em cada banco.

Código-fonte 23: Funções de medição de espaço em disco.

```
1 def medir_espaco_dados_pg(nome_tabela):
2     with engine_pg.connect() as conn:
3         return conn.execute(text(
4             f"SELECT pg_table_size('{nome_tabela}')")).scalar() /
      1024 / 1024
5
6 def medir_espaco_indice_pg(nome_tabela):
7     with engine_pg.connect() as conn:
8         tamanho = conn.execute(text(
9             f"SELECT COALESCE(SUM(pg_relation_size(indexrelid)), 0)::
      bigint "
```

```
10              f"FROM pg_index WHERE indrelid = '{nome_tabela}'::
      regclass "
11              f"AND NOT indisprimary")).scalar()
12      return tamanho / 1024 / 1024
13
14 def medir_espaco_dados_mongo(nome_colecao):
15      client_mongo.admin.command("fsync")
16      return db_mongo.command("collStats", nome_colecao)["storageSize"]
        / 1024 / 1024
17
18 def medir_tamanho_logico_mongo(nome_colecao):
19      return db_mongo.command("collStats", nome_colecao)["size"] / 1024
        / 1024
20
21 def medir_espaco_indice_mongo(nome_colecao):
22      client_mongo.admin.command("fsync")
23      stats = db_mongo.command("collStats", nome_colecao)
24      return (stats["totalIndexSize"] - stats["indexSizes"].get("_id_",
        0)) / 1024 / 1024
```

Nos dois bancos, o espaço é medido em disco e expresso em mebibytes (MiB). No PostgreSQL, o tamanho dos dados vem de `pg_table_size`, que considera a tabela e a área TOAST. Já o tamanho dos índices é a soma dos tamanhos dos índices da tabela, sem o índice da chave primária. Como todos os demais índices são removidos antes de cada medição, o valor obtido corresponde apenas ao índice criado para a consulta em teste.

No MongoDB, o tamanho dos dados vem do campo `storageSize` do comando `collStats`, que reflete o espaço efetivamente alocado em disco, já com a compressão aplicada pelo *WiredTiger*. Em seguida, mede-se também o tamanho sem compressão (campo `size`), para que a comparação com o PostgreSQL seja feita nas mesmas condições. O tamanho dos índices vem do campo `totalIndexSize`. O índice da chave primária (`_id`) é descontado, pois existe em todos os cenários e não representa o custo do índice em teste.

O espaço dos dados é medido uma vez após cada carga. O espaço ocupado pelos índices é reportado apenas nas consultas em que há um índice criado para o teste (C1, C4, C5, C6 e C7). As consultas C2, C3 e C8 localizam o item pelo identificador, que já é coberto pelo índice da chave primária nos dois bancos.

3.2.8 Experimentos de Escalabilidade

Para observar como cada operação se comporta quando o cenário cresce, foram definidos três experimentos de escalabilidade, cada um variando um fator e mantendo os demais fixos: o volume total do banco, a seletividade do filtro e o tamanho do lote de escrita. Os resultados de cada experimento são gravados em arquivos JSON para análise posterior.

3.2.8.1 Volume total do banco

Este experimento mede como cada operação se comporta quando o banco inteiro cresce. A pergunta é se, ao buscar exatamente a mesma coisa, o tempo piora e, em caso afirmativo, se piora na mesma proporção do crescimento dos dados. O banco é avaliado em três volumes, apresentados na Tabela 14. Para cada volume, os bancos são recriados do zero e carregados com dados gerados com a mesma semente, de modo que o conteúdo é reproduzível.

Tabela 14 – Volumes de dados avaliados.

Volume	Restaurantes	Itens
Pequeno	50	5.000
Médio	200	100.000
Grande	1.000	1.000.000

Fonte: Elaborado pela autora.

As consultas e os parâmetros são sempre os mesmos, o que muda de uma rodada para outra é apenas o tamanho do banco. Com isso, a proporção de registros que satisfazem cada filtro permanece aproximadamente constante.

3.2.8.2 Seletividade

A seletividade indica a fração dos registros que satisfaz o filtro de uma consulta. Este experimento mantém o volume do banco fixo (Grande) e varia o valor filtrado, de modo que a mesma consulta retorne quantidades diferentes de registros. A quantidade de linhas retornadas é registrada em cada medição, pois a seletividade depende dos dados gerados.

Foram escolhidas as consultas de leitura cujo filtro possui um parâmetro que controla o tamanho do resultado: C1, C4, C5 e C7, mostrado na Tabela 15. As outras consultas não foram incluídas por não terem um parâmetro variável ou pelo desejo de visualizar apenas um valor muito seletivo, como C6, que retorna apenas os itens com desconto de 80%.

Tabela 15 – Parâmetros variados no experimento de seletividade.

Consulta	Parâmetro variado	Valores
C1	Categoria do item	Sobremesa (cerca de 10% dos itens), bebida (cerca de 20%) e pizza (cerca de 35%)
C5	Largura da faixa de preço	R$ 5–12, R$ 5–25, R$ 5–50 e R$ 5–70
C4	Porte do restaurante	Pequeno, médio e grande (restaurantes que concentram quantidades crescentes de itens)
C7	Opção buscada	Picles extra (cerca de 0,09% dos itens), Cheddar extra (cerca de 9,4%) e Bacon extra (cerca de 20,5%)

Fonte: Elaborado pela autora.

O objetivo é observar como o tempo responde ao aumento do número de registros retornados.

3.2.8.3 Lote de escrita

Este experimento verifica se o custo *por item* diminui conforme o lote cresce e a partir de que tamanho essa melhoria deixa de ocorrer. O volume do banco é fixo (Grande), e os lotes variam entre 1, 10, 100, 1.000, 10.000, 50.000 e 100.000 itens. A Tabela 16 apresenta as operações avaliadas.

Tabela 16 – Operações avaliadas no experimento de lote de escrita.

Consulta	**Operação em lote**
C2 (inserção)	*Upsert* de *N* itens que ainda não existem, de modo que todos são inseridos
C2 (atualização)	*Upsert* de *N* itens que já existem, de modo que todos são atualizados
C3	Exclusão lógica (`status = 'DELETED'`) de *N* itens
C8	Adição da tag novidade a *N* itens já existentes

Fonte: Elaborado pela autora.

Nos experimentos C3 e C8, os identificadores dos itens do lote são sorteados entre os itens que já existem no banco. No C2, são gerados identificadores novos, pois o *upsert* insere itens que ainda não existem. Em todos os casos, o lote inteiro é enviado ao banco em uma única operação. No PostgreSQL, essa operação é uma instrução SQL seguida de *commit* e no MongoDB, é um `bulk_write` (C2) ou um `update_many` com `$in` (C3 e C8).

Para que as repetições executem o mesmo trabalho, utiliza-se duas funções auxiliares:

- `antes`: é o parâmetro opcional da função `medir_tempo` (Seção 3.2.7.1), executado antes de cada repetição e fora da cronometragem, em cada consulta ele desempenha a seguinte função:
  - C2 (inserção): os itens do lote são apagados e são gerados novos eventos.
  - C2 (atualização): os itens são criados uma vez, antes das repetições, e a cada repetição recebem evento novos, com versão maior e conteúdo diferente, de modo que a atualização condicional seja de fato executada.
  - C3 e C8: os campos alterados voltam aos valores originais.

  Depois dessa preparação, o cache dos bancos e do sistema operacional é descartado, como nos demais experimentos, de modo que cada repetição parte do cache vazio.
- `depois`: é executado uma única vez, ao fim das dez repetições de cada tamanho de lote, e devolve o banco ao estado original, de modo que o lote seguinte parta das mesmas condições.

Além do tempo total do lote, calcula-se o tempo por item, dado pela razão entre o tempo médio do lote e o número de itens *N*. É importante ressaltar que o tamanho do lote não trata de vários clientes simultâneos e sim de um único cliente enviando vários itens de uma vez.

3.3 Resultados Obtidos

Dar uma introdução de como ler os resultados

3.3.1 Efeito do Volume e do Índice

3.3.1.1 Consultas por id

Utilizar C3 para exemplificar

3.3.1.2 Consultas Seletivas

Utilizar C6 para exemplificar

3.3.1.3 Consultas pouco Seletivas

Utilizar C1 para exemplificar

3.3.1.4 Síntese do Ganho do Índice

Grafico 2

3.3.2 Seletividade

3.3.3 Lote

3.3.4 Carga Inicial dos Dados

3.3.5 Armazenamento

Nesta seção descreva o processo de obtenção dos resultados obtidos e analise-os à luz dos objetivos iniciais, bem como dos eventuais usuários (humanos ou não) de tais resultados. Preferencialmente, utilize critérios estabelecidos na área de pesquisa de seu projeto. Discuta sobre critérios utilizados para a validação do sistema, bem como os testes utilizados (se for o caso).

3.4 Dificuldades e Limitações

Nesta seção, descreva as principais dificuldades e limitações encontradas durante a condução do trabalho. Sintetize lições aprendidas e comente sobre direções alternativas, se for o caso. Se pertinente, faça uma análise crítica da abordagem adotada em seu projeto, ou seja, você a considera adequada? Ela é limitada sob algum aspecto? Indique quais pontos, decorrentes dos resultados, ou não, poderão ou deverão ser abordados por trabalhos futuros.

3.5 Considerações Finais

Nesta seção o aluno deve apresentar uma conclusão sobre este capítulo e introduzir brevemente o capítulo seguinte.

Capítulo 4

CONCLUSÃO

Este capítulo não deve ultrapassar 1 página, sendo que se deve comentar se os objetivos foram cumpridos, quais os desafios para cumprir os objetivos e principalmente o sentimento do aluno em relação à transposição das dificuldades. Ainda, vale salientar qual é o principal produto do trabalho. A última frase deve ser sobre a continuidade do trabalho.

REFERÊNCIAS

AGRAWAL, R.; SOMANI, A.; XU, Y. Storage and querying of e-commerce data. In: **Proceedings of the 27th International Conference on Very Large Data Bases (VLDB 2001)**. Roma, Italy: Morgan Kaufmann, 2001. p. 149–158. Citado na página 21.

AJI, R. F.; UTAMI, N. P. Evaluating field flexibility approaches in relational databases: A performance study of JSON and column-oriented models in library systems. **International Journal of Advanced Computer Science and Applications**, v. 17, n. 1, p. 37, 2026. Citado na página 26.

ASIMINIDIS, C.; KOKKONIS, G.; KONTOGIANNIS, S. Managing IoT data using relational schema and JSON fields, a comparative study. **IOSR Journal of Computer Engineering**, v. 20, n. 6, p. 46–52, nov. 2018. Disponível em: <https://www.iosrjournals.org/iosr-jce/papers/Vol20-issue6/Version-3/H2006034652.pdf>. Acesso em: 7 out. 2026. Citado na página 28.

BARTUNOV, O.; SIGAEV, T. **One step forward true json data type: nested hstore with arrays support**. 2013. Apresentação em PGCon 2013, Ottawa, 24 maio 2013. Disponível em: <https://www.pgcon.org/2013/schedule/attachments/280_hstore-pgcon-2013.pdf>. Acesso em: 7 out. 2026. Citado na página 21.

BONDUGULA, V. K. Storage optimization in distributed environments using optimistic concurrency control. **International Journal on Science and Technology**, v. 15, n. 2, p. 4755, 2024. Citado na página 26.

CHASSEUR, C.; LI, Y.; PATEL, J. M. Enabling JSON document stores in relational systems. In: **Proceedings of the 16th International Workshop on the Web and Databases (WebDB 2013)**. New York, NY, USA: ACM, 2013. p. 1–6. Citado na página 21.

DAVOUDIAN, A.; CHEN, L.; LIU, M. A survey on NoSQL stores. **ACM Computing Surveys**, v. 51, n. 2, p. 1–43, 2018. Citado na página 25.

DURNER, D.; LEIS, V.; NEUMANN, T. JSON tiles: Fast analytics on semi-structured data. In: **Proceedings of the 2021 International Conference on Management of Data**. New York: ACM, 2021. p. 445–458. Citado na página 22.

ELDAHSHAN, K.; SELIM, E.; EBADA, A. I.; ABOUHAWWASH, M.; NAM, Y.; BEHERY, G. Handling big data in relational database management systems. **Computers, Materials & Continua**, v. 72, n. 3, p. 5149–5164, 2022. Citado na página 21.

IRSHAD, L.; YAN, L.; MA, Z. Schema-based JSON data stores in relational databases. **Journal of Database Management**, v. 30, n. 3, p. 38–70, 2019. Citado na página 27.

LIU, Z. H.; HAMMERSCHMIDT, B.; MCMAHON, D.; CHANG, H.; LU, Y.; SPIEGEL, J.; SOSA, A. C.; SURESH, S.; ARORA, G.; ARORA, V. Native JSON datatype support. **Proceedings of the VLDB Endowment**, v. 13, n. 12, p. 3059–3071, 2020. Citado na página 26.

MATCHA, S.; MISHRA, R. Database selection and management: Choosing the right database (SQL vs. NoSQL) for your application. **International Journal of Research in Humanities and Social Sciences**, v. 13, n. 3, p. 68–88, mar. 2025. Citado na página 25.

MongoDB, Inc. **MongoDB Manual: FAQ: Concurrency**. 2026. Disponível em: <https://www.mongodb.com/docs/manual/faq/concurrency/>. Acesso em: 7 out. 2026. Citado na página 27.

NAYAK, A.; PORIYA, A.; POOJARY, D. Type of NoSQL databases and its comparison with relational databases. **International Journal of Applied Information Systems**, v. 5, n. 4, p. 16–19, mar. 2013. Disponível em: <https://www.ijais.org/archives/volume5/number4/434-0888/>. Acesso em: 7 out. 2026. Citado na página 26.

NGUYEN, T.-D.; LEE, S.-W. I/O characteristics of MongoDB and trim-based optimization in flash SSDs. In: **Proceedings of the Sixth International Conference on Emerging Databases: Technologies, Applications, and Theory**. New York: ACM, 2016. p. 139–144. Citado na página 27.

ORTÍZ, Y. V.; PIERRE, L. M.; LEÓN, A. R. S. Semi-structured data management in PostgreSQL: competing with MongoDB's performance. In: **Proceedings of the 10th Alberto Mendelzon International Workshop on Foundations of Data Management**. Aachen: CEUR-WS.org, 2016. (CEUR Workshop Proceedings, v. 1644). Disponível em: <https://ceur-ws.org/Vol-1644/paper31.pdf>. Acesso em: 7 out. 2026. Citado na página 28.

TRUICĂ, C.-O.; BOICEA, A.; TRIFAN, I. CRUD operations in MongoDB. In: **International Conference on Advanced Computer Science and Electronics Information (ICACSEI 2013)**. Paris: Atlantis Press, 2013. Citado na página 27.

URNIKIENĖ, J.; STEPONAVIČIENĖ, V.; ATANASOV, S. Comparative read performance analysis of PostgreSQL and MongoDB in e-commerce: an empirical study of filtering and analytical queries. **Big Data and Cognitive Computing**, v. 10, n. 2, p. 66, 2026. Citado na página 29.

WATERPEACE, S.; IVANOV, N. SoK: Evolution, security, and fundamental properties of transactional systems. In: **Proceedings of the 31st ACM Symposium on Access Control Models and Technologies (SACMAT '26)**. New York: ACM, 2026. p. 223–235. Citado na página 25.

APÊNDICE A

EXEMPLO DE APÊNDICE

Observação 1: Apêndice consiste em um texto ou documento elaborado pelo autor, a fim de complementar sua argumentação, sem prejuízo da unidade nuclear do trabalho. Os apêndices são identificados por letras maiúsculas consecutivas, travessão e pelos respectivos títulos. Observação 2: Elemento opcional. Observação 3: Se houverem mais apêndices, identifique-os como Apêndice B, Apêndice C e assim por diante.

Observação 1: Apêndice consiste em um texto ou documento elaborado pelo autor, a fim de complementar sua argumentação, sem prejuízo da unidade nuclear do trabalho. Os apêndices são identificados por letras maiúsculas consecutivas, travessão e pelos respectivos títulos. Observação 2: Elemento opcional. Observação 3: Se houverem mais apêndices, identifique-os como Apêndice B, Apêndice C e assim por diante.

ANEXO A

TÍTULO DO ANEXO

Observação 1: Anexos consistem em um texto ou documento não elaborado pelo autor, que serve de fundamentação, comprovação e ilustração. Os anexos são identificados por letras maiúsculas consecutivas, travessão e pelos respectivos títulos. Observação 2: Elemento opcional. Observação 3: Se houverem mais anexos, identifique-os como Anexo B, Anexo C e assim por diante.